

Doubly Linked List

Activity 1: Insert a new node into an empty list

```
#include <stdio.h>
#include <stdlib.h>

struct node{
    struct node* prev;
    int data;
    struct node* next;
};

struct node* addToEmpty(struct node* head, int data){
    struct node* temp = malloc(sizeof(struct node));
    temp->prev = NULL;
    temp->data = data;
    temp->next = NULL;
    head = temp;
    return head;
}

int main() {
    struct node* head = NULL;
    head = addToEmpty(head, 45);
    printf("%d", head->data);
    return 0;
}
```

Activity 2: Insert a new node into an existing list at the beginning.

```
#include <stdio.h>
#include <stdlib.h>

struct node{
    struct node* prev;
    int data;
    struct node* next;
};
```

```

struct node* addToEmpty(struct node* head, int data){
    struct node* temp = malloc(sizeof(struct node));
    temp->prev = NULL;
    temp->data = data;
    temp->next = NULL;
    head = temp;
    return head;
}

```

```

struct node* InsertAtBeginning(struct node* head, int data){
    struct node* temp = malloc(sizeof(struct node));
    temp->prev = NULL;
    temp->data = data;
    temp->next = NULL;
    temp->next = head;
    temp->prev = temp;
    head = temp;
    return head;
}

```

```

void displayForward(struct node* head) {
    printf("Forward traversal: ");
    while (head != NULL) {
        printf("%d -> ", head->data);
        head = head->next;
    }
    printf("NULL\n");
}

```

```

int main() {
    struct node* head = NULL;
    head = addToEmpty(head, 45);
    head = InsertAtBeginning(head, 35);
    displayForward(head);
    return 0;
}

```

Activity 3: Insert a new node into an existing list at the end.

```
#include <stdio.h>
#include <stdlib.h>

struct node{
    struct node* prev;
    int data;
    struct node* next;
};

struct node* addToEmpty(struct node* head, int data){
    struct node* temp = malloc(sizeof(struct node));
    temp->prev = NULL;
    temp->data = data;
    temp->next = NULL;
    head = temp;
    return head;
}

struct node* InsertAtBeginning(struct node* head, int data){
    struct node* temp = malloc(sizeof(struct node));
    temp->prev = NULL;
    temp->data = data;
    temp->next = NULL;
    temp->next = head;
    temp->prev = temp;
    head = temp;
    return head;
}

void displayForward(struct node* head) {
    printf("Forward traversal: ");
    while (head != NULL) {
        printf("%d -> ", head->data);
        head = head->next;
    }
    printf("NULL\n");
}

struct node* InsertAtEnd(struct node* head, int data){
```

```
    struct node* temp, *tp;
    temp = malloc(sizeof(struct node));
    temp->prev = NULL;
    temp->data = data;
    temp->next = NULL;
    tp = head;
    while (tp->next != NULL)
        tp = tp->next;
    tp->next = temp;
    temp->prev = tp;
    return head;
}
```

```
int main() {
    struct node* head = NULL;
    head = addToEmpty(head, 45);
    head = InsertAtBeginning(head, 35);
    head = InsertAtEnd(head, 25);
    displayForward(head);
    return 0;
}
```

Activity 4: Insert a new node into an existing list at a specific position.

```
#include <stdio.h>
#include <stdlib.h>

struct node{
    struct node* prev;
    int data;
    struct node* next;
};

struct node* addToEmpty(struct node* head, int data){
    struct node* temp = malloc(sizeof(struct node));
    temp->prev = NULL;
    temp->data = data;
    temp->next = NULL;
    head = temp;
    return head;
}

struct node* InsertAtBegining(struct node* head, int data){
    struct node* temp = malloc(sizeof(struct node));
    temp->prev = NULL;
    temp->data = data;
    temp->next = NULL;
    temp->next = head;
    temp->prev = temp;
    head = temp;
    return head;
}

void displayForward(struct node* head) {
    printf("Forward traversal: ");
    while (head != NULL) {
        printf("%d -> ", head->data);
        head = head->next;
    }
    printf("NULL\n");
}

struct node* InsertAtEnd(struct node* head, int data){
```

```

struct node* temp, *tp;
temp = malloc(sizeof(struct node));
temp->prev = NULL;
temp->data = data;
temp->next = NULL;
tp = head;
while (tp->next != NULL)
tp = tp->next;
tp->next = temp;
temp->prev = tp;
return head;
}

```

```

struct node* InsertAtPosition(struct node* head, int data, int position){

```

```

    struct node*newP = NULL;
    struct node*temp = head;
    struct node*temp2 = NULL;

```

```

    newP = addToEmpty(newP, data);

```

```

    while (position != 1){
        temp = temp -> next;
        position--;
    }

```

```

    if(temp->next == NULL)
    {
        temp->next = newP;
        newP->prev = temp;
    }

```

```

    else{
        temp2 = temp->next;
        temp->next = newP;
        temp2->prev = newP;
        newP->next = temp2;
        newP->prev = temp;
    }

```

```

    return head;

```

```

}

```

```

int main() {
    struct node* head = NULL;
    int position = 2;
    head = addToEmpty(head, 45);
    head = InsertAtBeginning(head, 35);
    head = InsertAtEnd(head, 25);
    head = InsertAtPosition(head, 15, position);
    displayForward(head);
    return 0;
}

```

Activity 5: Delete a node at the beginning.

```

#include <stdio.h>
#include <stdlib.h>

```

```

struct node{
    struct node* prev;
    int data;
    struct node* next;
};

```

```

struct node* addToEmpty(struct node* head, int data){
    struct node* temp = malloc(sizeof(struct node));
    temp->prev = NULL;
    temp->data = data;
    temp->next = NULL;
    head = temp;
    return head;
}

```

```

struct node* InsertAtBeginning(struct node* head, int data){
    struct node* temp = malloc(sizeof(struct node));
    temp->prev = NULL;
    temp->data = data;
    temp->next = NULL;
    temp->next = head;
    temp->prev = temp;
    head = temp;
    return head;
}

```

```

}
void displayForward(struct node* head) {
printf("Forward traversal: ");
while (head != NULL) {
printf("%d -> ", head->data);
head = head->next;
}
printf("NULL\n");
}

```

```

struct node* InsertAtEnd(struct node* head, int data){
    struct node* temp, *tp;
    temp = malloc(sizeof(struct node));
    temp->prev = NULL;
    temp->data = data;
    temp->next = NULL;
    tp = head;
    while (tp->next != NULL)
    tp = tp->next;
    tp->next = temp;
    temp->prev = tp;
    return head;
}

```

```

struct node* InsertAtPosition(struct node* head, int data, int position){

```

```

    struct node*newP = NULL;
    struct node*temp = head;
    struct node*temp2 = NULL;

```

```

    newP = addToEmpty(newP, data);

```

```

    while (position != 1){
        temp = temp -> next;
        position--;
    }

```

```

    if(temp->next == NULL)
    {
        temp->next = newP;
        newP->prev = temp;
    }

```



```

else{
    temp2 = temp->next;
    temp->next
    = newP;
    temp2->prev = newP;
    newP->next = temp2;
    newP->prev = temp;
}
return head;
}

```

```

struct node* deleteAtBeginning(struct node* head)
{
    struct node* temp = head;
    head = head ->next;
    free(temp);
    temp = NULL;
    head ->prev = NULL;
    return head;
}

```

```

int main() {
    struct node* head = NULL;
    int position = 2;
    head = addToEmpty(head, 45);
    head = InsertAtBeginning(head, 35);
    head = InsertAtEnd(head, 25);
    head = InsertAtPosition(head, 15, position);
    head = deleteAtBeginning(head);
    displayForward(head);
    return 0;
}

```

Activity 6: Delete a node at the end.

```
#include <stdio.h>
#include <stdlib.h>

struct node{
    struct node* prev;
    int data;
    struct node* next;
};

struct node* addToEmpty(struct node* head, int data){
    struct node* temp = malloc(sizeof(struct node));
    temp->prev = NULL;
    temp->data = data;
    temp->next = NULL;
    head = temp;
    return head;
}

struct node* InsertAtBeginning(struct node* head, int data){
    struct node* temp = malloc(sizeof(struct node));
    temp->prev = NULL;
    temp->data = data;
    temp->next = NULL;
    temp->next = head;
    temp->prev = temp;
    head = temp;
    return head;
}

void displayForward(struct node* head) {
    printf("Forward traversal: ");
    while (head != NULL) {
        printf("%d -> ", head->data);
        head = head->next;
    }
    printf("NULL\n");
}

struct node* InsertAtEnd(struct node* head, int data){
```

```

struct node* temp, *tp;
temp = malloc(sizeof(struct node));
temp->prev = NULL;
temp->data = data;
temp->next = NULL;
tp = head;
while (tp->next != NULL)
tp = tp->next;
tp->next = temp;
temp->prev = tp;
return head;
}

```

```

struct node* InsertAtPosition(struct node* head, int data, int position){

```

```

    struct node*newP = NULL;
    struct node*temp = head;
    struct node*temp2 = NULL;

```

```

    newP = addToEmpty(newP, data);

```

```

    while (position != 1){
        temp = temp -> next;
        position--;
    }

```

```

    if(temp->next == NULL)
    {
        temp->next = newP;
        newP->prev = temp;
    }

```

```

    else{
        temp2 = temp->next;
        temp->next
        = newP;
        temp2->prev = newP;
        newP->next = temp2;
        newP->prev = temp;
    }

```

```

    return head;

```

```

}

```

```

struct node* deleteAtBeginning(struct node* head)
{
    struct node* temp = head;
    head = head ->next;
    free(temp);
    temp = NULL;
    head ->prev = NULL;
    return head;
}

```

```

struct node* deleteAtEnd(struct node* head)
{
    struct node* temp = head;
    struct node* temp2;

    while (temp->next != NULL)
        temp = temp->next;
    temp2 = temp->prev;
    temp2->next = NULL;
    free(temp);
    temp = NULL;
    return head;
}

```

```

int main() {
    struct node* head = NULL;
    int position = 2;
    head = addToEmpty(head, 45);
    head = InsertAtBeginning(head, 35);
    head = InsertAtEnd(head, 25);
    head = InsertAtPosition(head, 15, position);
    head = deleteAtBeginning(head);

    head = deleteAtEnd(head);
    displayForward(head);
    return 0;
}

```

Activity 6: Delete a node at some specific position.